Universidad Veracruzana
Facultad de Estadística e Informática · Región Xalapa
Lic. en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Programación en Red — Unidad II · Programación a nivel de sockets

Concurrencia con hilos: teoría y práctica con TCP y UDP

Guía de clase · Práctica 6 · Viernes 02 de octubre de 2026
Mtro. José Pablo Rueda Holguín · NRC 19141

1. Objetivos de la sesión

- Comprender qué es un hilo y en qué se diferencia de un proceso.
- Programar concurrencia en Python con el módulo threading.
- Aplicar hilos en un servidor TCP para atender varios clientes de forma simultánea.
- Aplicar UDP para consultas ligeras sin establecer conexión.
- Sentar las bases de la Práctica 6: sistema de turnos LA FILA.

2. Fundamento teórico: concurrencia e hilos

2.1 Proceso e hilo

Un **proceso** es un programa en ejecución con su propio espacio de memoria: sus variables no son visibles para otros procesos. Un **hilo (thread)** es una línea de ejecución independiente dentro de un proceso: todos los hilos de un mismo proceso comparten la misma memoria, es decir, ven las mismas variables globales.

Crear un hilo es mucho más económico que crear un proceso, y la comunicación entre hilos es directa precisamente porque comparten memoria. Esa ventaja tiene un costo: si dos hilos modifican la misma variable al mismo tiempo, el resultado puede ser incorrecto (condición de carrera), por lo que el acceso a datos compartidos debe controlarse.

2.2 ¿Por qué hilos en programación en red?

Las operaciones de red son operaciones de entrada/salida **bloqueantes**: cuando un programa llama a recv(), la ejecución se detiene hasta que llegan datos. Si el servidor atiende con un solo hilo, un cliente que tarde en responder detiene a todos los demás.

La solución es el patrón **un hilo por conexión**: el hilo principal del servidor solo acepta conexiones nuevas y, por cada cliente, crea un hilo que lo atiende. Mientras un hilo espera datos de su cliente, los demás hilos continúan trabajando. En Python, los hilos no aceleran el cómputo intensivo de procesador (por el GIL), pero son adecuados cuando el tiempo se pasa esperando entrada/salida, que es exactamente el caso de los sockets.

2.3 Reglas de uso de los hilos

- **Compartido significa riesgoso:** toda variable global que varios hilos modifiquen (por ejemplo, un contador) es una sección crítica. Se protege con threading.Lock.
- **daemon=True:** los hilos de atención a clientes se crean como daemon para que no impidan que el servidor termine.
- **start() y join():** start() arranca el hilo y regresa de inmediato; join() bloquea al hilo que la llama hasta que el hilo indicado termina.
- **Salidas entrelazadas:** cuando varios hilos imprimen, el orden de los mensajes en pantalla puede variar de una ejecución a otra. Es el comportamiento esperado.

3. El módulo threading: hola mundo con hilos

El siguiente programa crea dos hilos que ejecutan la misma función con argumentos distintos. Es el ejemplo mínimo para observar la concurrencia: ejecútelo varias veces y compare el orden de los mensajes.

Código 1 · hola_mundo_hilos.py

```
1  import threading
2  import time
3
4
5  def saludar(nombre):
6      """Imprime tres saludos con una pausa de un segundo."""
7      for i in range(3):
8          print(f"{nombre}: hola {i}")
9          time.sleep(1)
10
11
12 def main():
13     hilo_ana = threading.Thread(target=saludar, args=("Ana",))
14     hilo_beto = threading.Thread(target=saludar, args=("Beto",))
15     hilo_ana.start()
16     hilo_beto.start()
17     hilo_ana.join()
18     hilo_beto.join()
19     print("Fin del programa")
20
21
22 main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
1–2	import threading, time	threading crea y administra hilos; time.sleep(1) pausa la ejecución un segundo para hacer visible el entrelazado.
5–9	def saludar(nombre)	Función que ejecutará cada hilo. Recibe un nombre y escribe tres saludos con una pausa entre ellos.
13–14	threading.Thread(target=saludar, args=("Ana",))	Crea el hilo: target es la función a ejecutar (se escribe sin paréntesis) y args es una tupla con sus argumentos; la coma en ("Ana",) es obligatoria para que sea tupla.

Línea(s)	Elemento	Explicación
15–16	start()	Arranca el hilo y regresa de inmediato. A partir de aquí, los dos hilos y el programa principal avanzan de forma concurrente.
17–18	join()	El programa principal espera a que cada hilo termine antes de continuar; sin join(), el mensaje final podría aparecer antes que los saludos.
19	print("Fin del programa")	Gracias a los join(), este mensaje siempre se imprime al final.

Qué observar: los saludos de Ana y Beto aparecen entrelazados y el orden puede variar entre ejecuciones; "Fin del programa" siempre aparece al final.

4. Ejemplo con TCP: servidor de eco multicliente

Este ejemplo aplica el patrón **un hilo por conexión** sobre TCP: el servidor acepta clientes en el puerto 5000 y crea un hilo para atender a cada uno, de modo que varios clientes puedan estar conectados al mismo tiempo. Los mensajes viajan en texto plano con framing por salto de línea (\n), igual que en las prácticas anteriores.

Código 2 · servidor_eco_tcp.py

```python
import socket
import threading


def atender_cliente(conexion, direccion):
    """Reenvía al cliente cada línea que recibe (servidor de eco)."""
    with conexion:
        buffer = ""
        while True:
            datos = conexion.recv(1024)
            if not datos:
                break
            buffer += datos.decode("utf-8")
            while "\n" in buffer:
                linea, buffer = buffer.split("\n", 1)
                conexion.sendall((linea + "\n").encode("utf-8"))
    print(f"Conexión cerrada: {direccion}")


def main():
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    servidor.bind(("0.0.0.0", 5000))
    servidor.listen()
    print("Servidor TCP escuchando en el puerto 5000")
    while True:
        conexion, direccion = servidor.accept()
        hilo = threading.Thread(target=atender_cliente,
                                args=(conexion, direccion),
                                daemon=True)
        hilo.start()


main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
1–2	import socket, threading	socket para la comunicación TCP; threading para atender cada cliente en su propio hilo.
5–7	def atender_cliente(conexion, direccion)	Función que ejecuta cada hilo. Recibe el socket de la conexión y la dirección del cliente. with conexion garantiza que el socket se cierre al salir.
8–12	buffer, while y recv(1024)	recv() es bloqueante: espera datos. Si devuelve bytes vacíos, el cliente cerró la conexión y el hilo termina su ciclo con break.
13–16	framing y sendall()	TCP es un flujo de bytes sin fronteras de mensaje: se acumula en buffer y se extraen mensajes completos usando \n como delimitador. Cada línea recibida se reenvía al cliente (eco) con sendall().
17	print de cierre	Informa en consola qué cliente se desconectó.
21–24	socket, SO_REUSEADDR, bind, listen	Crea el socket TCP, permite reutilizar el puerto tras reiniciar el programa, lo enlaza a todas las interfaces (0.0.0.0) en el puerto 5000 y lo pone a escuchar.
26–27	accept()	Bloquea hasta que llega un cliente y devuelve (conexion, direccion). El hilo principal solo hace esto: aceptar.
28–31	Thread(..., daemon=True) y start()	Crea un hilo de atención por conexión y lo arranca. daemon=True permite que el servidor se cierre aunque queden hilos activos.

Código 3 · cliente_eco_tcp.py

```python
import socket


def main():
    with socket.create_connection(("127.0.0.1", 5000)) as conexion:
        for mensaje in ("hola", "redes", "adios"):
            conexion.sendall((mensaje + "\n").encode("utf-8"))
            respuesta = conexion.recv(1024).decode("utf-8").strip()
            print(f"Servidor responde: {respuesta}")


main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
5	create_connection	Abre la conexión TCP con el servidor (IP y puerto); with garantiza el cierre.
6–7	sendall(mensaje + \n)	Envía cada mensaje codificado en UTF-8 y terminado en \n, conforme al framing acordado.

Línea(s)	Elemento	Explicación
8	recv(1024).decode().strip()	Espera la respuesta del servidor, la decodifica y retira el salto de línea.

Para probar la concurrencia: ejecute el servidor y, desde dos o más terminales (o equipos), ejecute el cliente al mismo tiempo. Todos reciben su eco sin esperar a que otro termine.

5. Ejemplo con UDP: consulta sin conexión

UDP no establece conexión: cada datagrama es un mensaje completo e independiente. El servidor atiende las consultas en un ciclo con recvfrom(), que devuelve los datos y la dirección del remitente para poder responder con sendto(). Si el procesamiento de cada consulta fuera costoso, se lanzaría un hilo por datagrama; en este ejemplo basta el ciclo porque la respuesta es inmediata.

Código 4 · servidor_consulta_udp.py

```
1  import socket
2
3
4  def main():
5      servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6      servidor.bind(("0.0.0.0", 5001))
7      print("Servidor UDP escuchando en el puerto 5001")
8      while True:
9          datos, direccion = servidor.recvfrom(1024)
10         mensaje = datos.decode("utf-8").strip()
11         if mensaje == "ESTADO":
12             respuesta = "servidor activo"
13         else:
14             respuesta = "consulta no reconocida"
15         servidor.sendto((respuesta + "\n").encode("utf-8"), direccion)
16
17
18 main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
5	socket(AF_INET, SOCK_DGRAM)	Crea el socket UDP (SOCK_DGRAM), a diferencia del SOCK_STREAM de TCP.
6	bind(("0.0.0.0", 5001))	Enlaza el socket al puerto 5001. En UDP no hay listen() ni accept(): no existen conexiones.
9	recvfrom(1024)	Bloquea hasta recibir un datagrama y devuelve dos valores: los datos y la dirección (IP, puerto) de quien lo envió.
11–14	lógica de consulta	Interpreta el mensaje: si es ESTADO responde que el servidor está activo; en otro caso indica que no reconoció la consulta.
15	sendto(respuesta, direccion)	Envía la respuesta al remitente usando la dirección que devolvió recvfrom().

Código 5 · cliente_consulta_udp.py

```python
import socket


def main():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
        cliente.settimeout(3)
        cliente.sendto("ESTADO\n".encode("utf-8"), ("127.0.0.1", 5001))
        datos, _ = cliente.recvfrom(1024)
        print(f"Servidor responde: {datos.decode('utf-8').strip()}")


main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
6	settimeout(3)	Si la respuesta no llega en 3 segundos, recvfrom() lanza TimeoutError en lugar de esperar indefinidamente; en UDP no hay retransmisión.
7	sendto(mensaje, (ip, puerto))	Envía un datagrama al servidor. Un datagrama equivale a un mensaje completo.
8	recvfrom(1024)	Espera el datagrama de respuesta; se descarta la dirección con _ porque ya se conoce.

Observación: los puertos 5000 (TCP) y 5001 (UDP) son independientes; el mismo programa puede atender ambos a la vez, cada uno en su propio hilo, como se pedirá en la Práctica 6.

6. Práctica 6 — Sistema de turnos LA FILA

6.1 Caso real

Se programará la fila de una ventanilla de atención: cada cliente que llega **toma un turno** y el servidor asigna números consecutivos (1, 2, 3, …) sin repeticiones ni saltos. Varios clientes solicitan turno de forma simultánea, lo que justifica la concurrencia con hilos; además, cualquier equipo puede consultar cuántos turnos se han asignado sin establecer conexión, lo que justifica UDP.

6.2 Protocolo de aplicación

Canal	Mensaje	Comportamiento
TCP 5000	TURNO>apodo\n	El cliente se conecta y envía su apodo. El servidor responde turno>N\n con su número asignado.
UDP 5001	CUANTOS	El cliente envía un datagrama con la palabra CUANTOS. El servidor responde van>N\n con el total de turnos asignados.

Reglas de codificación: texto plano en UTF-8; en TCP cada mensaje termina en \n (framing de las prácticas anteriores); en UDP un datagrama equivale a un mensaje completo.

6.3 Reglas de la práctica

- **Trabajo en parejas:** cada pareja desarrolla su propio servidor y sus propios clientes.

- **Módulos permitidos:** socket, threading y time; no se permiten bibliotecas externas.
- **La validación se realiza entre dos equipos de cómputo** (no únicamente en localhost); se utiliza la dirección IP real del servidor.
- **Arquitectura mínima:** un hilo por cliente TCP conectado y un hilo para el ciclo de consultas UDP; el contador de turnos es una variable compartida y su incremento es una sección crítica.
- **Código limpio:** nombres descriptivos, una instrucción por línea y comentarios que expliquen el porqué de cada sección.

6.4 Entrega

Demostración en clase el **martes 06 de octubre de 2026**: dos clientes TCP toman turno al mismo tiempo y un tercer equipo consulta por UDP; el servidor debe asignar turnos consecutivos sin repetir ninguno. La lógica de asignación de turnos se construye en clase a partir de los Códigos 1 a 5 de esta guía.

7. Errores típicos y cómo evitarlos

Contexto	Error	Cómo evitarlo
TCP	Usar recv() como si devolviera un mensaje completo	TCP es un flujo continuo: acumular en un buffer y extraer mensajes por \n.
TCP	Olvidar el \n al enviar	Sin delimitador, el receptor no sabe dónde termina el mensaje y se queda esperando.
Ambos	Enviar cadenas sin codificar	sendall() y sendto() requieren bytes: usar encode("utf-8") al enviar y decode("utf-8") al recibir.
UDP	recvfrom() sin timeout en el cliente	Si el datagrama se pierde no hay retransmisión: usar settimeout() y manejar TimeoutError.
Hilos	Escribir args=("Ana") sin la coma	Sin la coma no es una tupla: la forma correcta es args=("Ana",).
Hilos	Llamar a la función en lugar de pasarla	target=saludar() ejecuta la función de inmediato; lo correcto es target=saludar.
Hilos	Contador compartido sin protección	Dos hilos pueden incrementar a la vez y repetir turno: proteger el incremento con threading.Lock.
Servidor	Address already in use al reiniciar	Falta setsockopt(SOL_SOCKET, SO_REUSEADDR, 1) antes del bind().